

Weekly Progress Report

Project: Custom 2D Platformer - Group 3

Date / Week: 19/7 to 25/7 | Week 07

1. Weekly Overview

- **Primary Goal:** Complete all delayed tasks from Week 4 to Week 7 to finalize the core mechanics, entity interactions, and UI integration.
- **Core Physics & Collision (Minh Khoa):** Implement the delayed gravity system and precise AABB collision detection. Integrate complex interactions (e.g., Mario stomping on Goombas) and refine hitboxes to perfectly match the 1985 gameplay feel.
- **Level Rendering & UI (Đặng Khoa):** Develop the tilemap renderer to display static Mario levels (bricks, pipes) from text files. Implement the Observer pattern to dynamically update the classic HUD (score, coins, time).
- **Hero & Items Logic (Khải):** Spawn classic items (Mushroom, Coin) into the game. Finalize Mario's advanced mechanics, including invincibility frames and Fire Mario state transitions.
- **Enemies & AI (Long):** Spawn Goombas and Koopas and implement their basic walking/bouncing behaviors. Finalize advanced enemy logic (e.g., Koopa shells sliding and hitting other enemies).
- **Completion Rate:** 75% of the weekly plan completed.

2. Individual Task Breakdown

25125057 - Trần Minh Khoa is responsible for the **Core Engine & Physics**.

- **Completed Tasks:**
 - **State Machine Expansion:** Implemented `TransitionState`, `GameOverState`, and `VictoryState` to manage the full gameplay lifecycle and handle UI screen transitions.
 - **Systems Decoupling:** Established a clear architectural boundary between the core game engine (screen states, physics computations) and entity logic (character animations, real-time keyboard inputs).
 - **Physics Engine Development:** Built a dedicated `PhysicsEngine` class to process vertical acceleration (gravity) and apply dynamic velocity using a fixed time-step (`dt`) for consistent frame rates.
 - **AABB Collision System:** Implemented Axis-Aligned Bounding Box (AABB) collision detection, utilizing a two-step (X-axis, then Y-axis) movement pipeline to accurately resolve floor/ceiling impacts and prevent horizontal wall clipping.
 - **Physics Prototyping:** Developed a visual testing sandbox within the `PlayingState` using dummy bounding boxes (`Sf::RectangleShape`) to independently validate gravity and directional collision

mathematics prior to the final entity integration.

- **Proof:**

- <https://github.com/trannhukhai98438/CS202-Group03-2DGame/commit/83587b1af0a715ea6c61a933d078b78ff8fcfc61>
- <https://github.com/trannhukhai98438/CS202-Group03-2DGame/commit/bce01de112a5a401cd218fd6beedee63da78167c>

25125045 - Trần Như Khải is in charge of the **Hero & Items Logic**.

- **In-process Tasks:**

- Refactoring and re-implementing the **Character** and **Hero** classes to improve the architecture.
- Researching and collecting appropriate sprite sheets for both Mario and Luigi characters.

25125024 - Đỗ Viết Hoàng Long is handling the **Enemies & AI Design**.

- **Completed Tasks:**

- **Base Enemy & Character Hierarchy:** Created the abstract **Character** base class and **Enemy** abstract class, establishing foundational inheritance for all game entities.
- **Patrol AI via Template Method Pattern:** Implemented standard patrol AI behavior in **Enemy::update()** using the Template Method Pattern, delegating **checkObstacles()**, **move()**, and **applyAnimation()** primitive steps to derived classes.
- **Classic Enemies Implementation:** Developed concrete **Goomba** (walking & squish mechanics) and **Koopa** (walking, shell transformation, and high-speed sliding shell mechanics).
- **Polymorphic AI State Management:** Built the **EnemyState** polymorphic state pattern hierarchy (**PatrolState**, **SquishedState**, **ShellState**, **SpinningShellState**) for dynamic AI behavior transitions.
- **Factory Pattern Implementation:** Implemented **EnemyFactory** (Simple Factory, String-based map loader creation, and Factory Method creators **ConcreteGoombaFactory**, **ConcreteKoopaFactory**).

- **Proof:**

- <https://github.com/trannhukhai98438/CS202-Group03-2DGame/commit/fbf38dea9c132b0f7e107b1a73dd589b80e778bc>

25125056 - Trần Đăng Khoa is assigned to the development of the **Tilemap, HUD & Audio**.

- **Completed Tasks:**

- Implemented LevelManagment class.

3. Challenges & Solutions

Issue 1: The **Mario** class was becoming overly bloated and difficult to maintain as it had to manage too many parameters and complex character states (e.g., Dead, Fire, Giant, Small). **Solution 1:** Abstracted the core functionalities into a general **Hero** class and redesigned the character's form logic using the **Strategy Pattern**. This allows the character to swap states dynamically and keeps the code clean and scalable.

4. AI Usage Notes

- Gemini. Gemini 3.1 Pro, Google, gemini.google.com, accessed 23:00 on July 24, 2026, prompt: "How do I implement the remaining lifecycle states (GameOverState, VictoryState, and TransitionState) in C++"

SFML, and does a TimePerFrame of 1/60th second guarantee exact state refresh rates?" to finalize the state machine architecture, AI suggested the implementation blueprints for the three states and explained the fixed time-step logic, Student reviewed the code and integrated the states into the engine.

- Gemini. Gemini 3.1 Pro, Google, gemini.google.com, accessed 01:00 on July 25, 2026, prompt: "Explain the line-by-line math behind AABB collision detection in SFML, including the inverted Y-axis subtraction, the .f syntax, and how to upgrade the PhysicsEngine to prevent X-axis wall clipping." to deeply understand and implement AABB physics, AI suggested a detailed mathematical breakdown of screen coordinates and a two-step movement pipeline for directional collision resolution, Student reviewed the explanation, built the PhysicsEngine class, and tested it using dummy hitboxes.
- Gemini. Gemini 3.6 Flash, Google, gemini.google.com, accessed 18:00 on July 25, 2026, prompt: "How do I structure the Enemy class header in C++ so it stays fully compatible with my teammate's Character class in feature/Hero branch without header include conflicts?" to check cross-teammate file dependencies, AI suggested using forward headers and relative includes, Student reviewed the header paths and implemented the compatibility layer.
- Gemini. Gemini 3.6 Flash, Google, gemini.google.com, accessed 19:30 on July 25, 2026, prompt: "What are the exact state transitions and gameplay mechanics for Goomba and Koopa in Super Mario Bros (stomping, squished duration, shell mode, and sliding shell velocity)?" to design realistic enemy behavior, AI explained the state transitions and movement parameters, Student implemented the Goomba and Koopa class methods.
- Gemini. Gemini 3.6 Flash, Google, gemini.google.com, accessed 20:00 on July 25, 2026, prompt: "Draw an OOP class hierarchy graph and design pattern diagram for Enemy, EnemyState, and EnemyFactory to apply Template Method and State patterns." to map out object relationships before coding, AI provided an object relationship blueprint, Student reviewed the diagram and wrote the C++ classes.
- Gemini. Gemini 3.6 Flash, Google, gemini.google.com, accessed 21:00 on July 25, 2026, prompt: "Format my completed task breakdown and proof links for Week 07 progress report following the course markdown template." to generate the weekly report documentation, AI formatted the detailed bullet points and GitHub commit URL, Student reviewed and integrated it into the week_07.md report.
- Gemini. Gemini 3.1 Pro, Google, gemini.google.com, accessed 22:05 on July 25, 2026, prompt: "How can State/Strategy Patterns be applied to the Hero for various character states?", to refactor the character's form logic, AI outlined the structural design and provided example interaction functions, Student referenced the structure to finalize the implementation of the Hero class.

5. Next Week's Action Plan

- **Core Mechanics:**
 - Integrate the newly developed **PhysicsEngine** (gravity and AABB collision) with actual game entities (Hero, Enemies) instead of testing with dummy bounding boxes.
 - Finalize Khải's in-process refactoring of the **Character** and **Hero** classes, fully implementing Mario's state-swapping logic via the Strategy Pattern.
 - Implement complex entity interaction logic (e.g., Mario squishing Goombas, colliding with sliding Koopa shells, or taking damage).
- **System Integration:** Merge all individual modules (Engine, Level Management, Hero, Enemies) into the main development branch to successfully assemble and run the first playable gameplay demo (All Members).
- **Gameplay / Graphics:**

- Develop the tilemap renderer to load and display static Mario levels (bricks, pipes) from text/csv files (delayed task).
- Implement the dynamic HUD (score, coins, time) utilizing the Observer Pattern.
- Integrate the completed **Goomba** and **Koopa** classes into the main game loop to test their Patrol AI and state management.
- Spawn classic items (Mushroom, Coin) utilizing the Factory Pattern.
- **Expected Deadline:** 01/8/2026